

AMT630HV100 BSP 用户说明

1 BSP 目录结构

目录名	描述
amt630hv100-freertos	应用工程文件，使用 IAR ARM 8.30 ^① 编译调试，基于 FreeRTOS +LVGL(AWTK)，支持 JTAG 调试
AMTLDR	升级、启动工程文件，使用 IAR ARM 8.30 编译调试，裸机程序
STEPLDR	二级加载工程文件，处理各种模式的升级，同样使用 IAR ARM 8.30，裸机程序
Doc	相关文档说明
Tools	开发中需要用到的各种工具
升级文件	SD 卡升级文件，LVGL 目录下包含使用 lvgl 和 openvg 来绘制界面的 demo；VG_ONLY 目录下包含只使用 openvg 来绘制界面的单表盘和双表盘 Demo；AWTK 目录下包含几个 AWTK 官方 Demo。

1.1 应用工程目录结构(主要内容)

目录名	描述
app	应用逻辑代码
ArkmicroFiles	板级以及芯片驱动代码
FreeRTOS	FreeRTOS 内核
FreeRTOS-Plus	FreeRTOS 扩展，包括 FAT，POSIX，CLI(命令行)等
lib	第三方库，包括 LVGL，AWTK，VG_DRIVER，SFUD(串行 flash 万能驱动库)等
proj	工程文件

2 SD 卡升级说明

2.1 升级方法

将升级文件目录下 LVGL 目录内 AMTLDR.bin、spildr.bin、stepldr.bin 和 update.bin 四个文件拷贝到 tf 卡后将卡插入开发板 tf 卡槽，并将板上启动模式跳帽跳到 BIT1(0):BIT0(0) SD+SPINOR 模式后上电，此时 LCD 屏上会显示升级进度条，调试串口会输

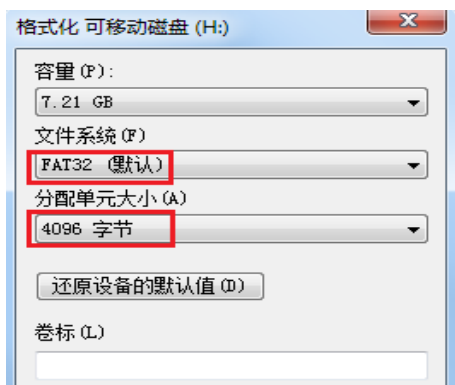
出以下信息，表示升级完成。升级完成后会自动进入应用系统，在 LCD 屏上依次显示开机 logo，开机动画，仪表界面。

```
ARK SoC Booter V 2.00
boot_media: : 0x00000000
SDMMC card successfully
Mount file ok
size: : 0x00004b24

ARK AMT 630Hv100 FROM SD V 1.21

DDR init over2!!
SDMMC card successfully
Mount file ok
load stepldr...
ARK AMT630H STEPLDR V 1.0
SDMMC card successfully
Mount file ok
burn loader start...
burn loader end.
burn stepldr start...
burn stepldr end.
burn app start...
burn app end.
update ok, save sysinfo.
update is finished.
```

需要注意的是升级卡需要**格式化**成 **FAT32** 文件系统，另外分配单元大小要小于或者等于 4096 字节，最好使用 4g~16g 容量的卡。



2.2 升级文件说明

AMTLDR.bin

卡升级程序，由 AMTLDR 工程编译生成，注意需要将工程目录内 Src/Entry.c 文件里 PROJECT_PURPOSE 宏定义的值改为 PROJECT_FOR_SD_UPDATE。

spildr.bin

spinor 引导程序，AMTLDR 工程编译生成，注意需要将工程目录内 Src/Entry.c 文件里

PROJECT_PURPOSE 宏定义的值改为 PROJECT_FOR_SPIFLASH_LOADER。之后还需要将编译生成的 AMTLDR.bin 文件重命名为 spildr.bin。

stepldr.bin

spinor 二级引导程序，STEPLDR 工程编译生成，主要实现各种模式的升级功能以及升级异常处理。

update.bin

应用升级文件，由 amt630hv100.bin、bootanim.bin 和 rom.bin 通过工具制作生成。

amt630hv100.bin

应用程序，amt630h-freertos 工程编译生成。

bootanim.bin

动画文件，具体制作方法参考 Doc 目录下《amt630hv100 开机动画文件制作说明》。注意因为升级程序不支持长文件名需要将 bootanimation 文件重命名为 bootanim.bin。

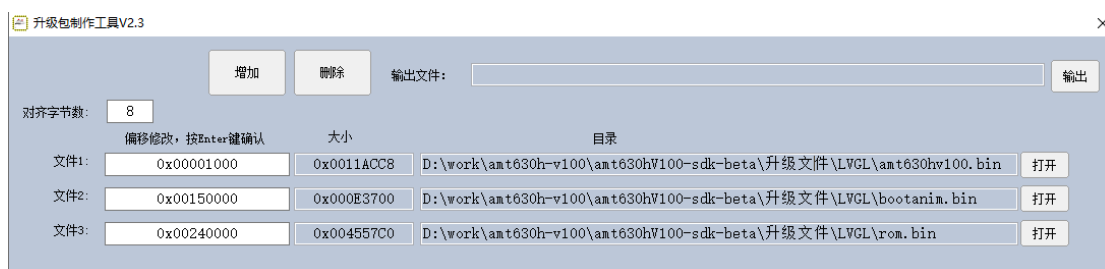
rom.bin

资源打包文件，将 Tool/RomMaker 目录下 rommaker.exe^①文件拷贝到资源文件所在的文件夹后，双击 rommaker.exe 运行，则会在当前目录下生成 rom.bin 文件。以资源文件所在的目录作为当前目录，所有文件相对于当前目录的相对路径名长度不能大于 64。

2.3 update.bin 文件制作说明

打开 Tools\630HV100 升级包工具\ 630HV100 升级包工具 V2.3.exe 应用，依次添加上述的 amt630hv100.bin、rom.bin 和 bootanim.bin 三个文件，之后点击输出生成 update.bin 文件。需要特别注意的是 amt630hv100.bin 文件必须作为第一个文件添加，另外每个文件的偏移必须按照 4KB(spinor flash sector erase size) 进行对齐，例如 0x1000(输入后需要按回车确认)。

出厂后机器支持串口升级或者后续会添加的 CAN 升级，并且支持 amt630hv100.bin、bootanim.bin 和 rom.bin 中某个文件单独升级。但是，由于单个文件升级时新的升级文件可能会大于之前的文件，如果之前三个文件都是紧挨着连续放置的话升级新的文件会覆盖其他文件的数据，所以在制作 update.bin 文件时最好提前预留一部分空间，即第二个文件的偏移设置为第一个文件的偏移加上第一个文件的大小后再加上预留的空间大小。动画文件 bootanim.bin 通常出厂后不需要升级，可以把它作为第二个文件，rom.bin 作为第三个文件，这样整个 spinor flash 剩余的空间都可以作为后续 rom.bin 升级时候的额外空间。

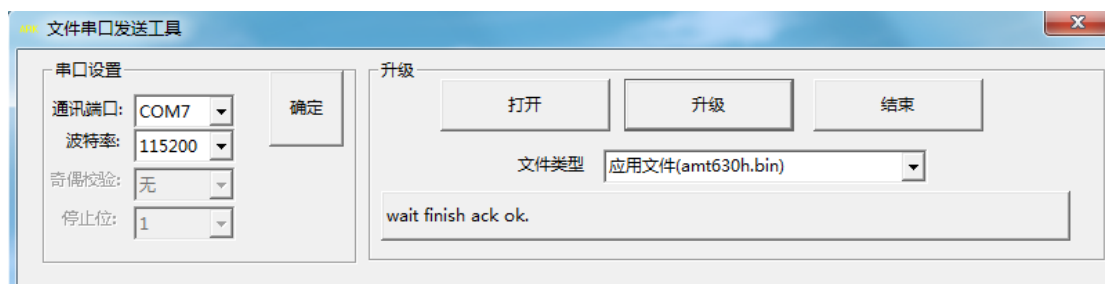


如上图所示，amt630hv100.bin 的文件偏移设置为 0x1000，bootanim.bin 文件偏移设置为 0x150000，这样可以确保后续单独升级 amt630hv100.bin 文件时该文件大小只要不超过 (0x150000 - 0x1000) 大小就行。rom.bin 放到最后，后续单独升级 rom.bin 文件大小只要在原有偏移的基础上不超过整个 spinor flash 大小就可以。

3 串口升级说明

串口升级支持升级完整的 update.bin 文件，也支持单独升级 amt630hv100.bin, rom.bin, bootanim.bin，特殊情况下也可以升级 spildr.bin 和 stepldr.bin。

串口升级时，等应用跑起来后，打开 Tools\630HV100 串口升级工具\630HV100 串口升级工具.exe 应用，如下图所示：



设置好串口，点击确定后，点击打开选择你要升级的文件，文件类型会自动匹配，如果升级文件是 amt630hv100.bin，需要再文件类型选项里手动选择“应用文件 (amt630h.bin)”，然后点击升级按钮开始升级，此时机器会重启然后在屏上开始显示升级进度条，升级结束后自动进入系统。

串口升级通信协议详见 Doc 目录下《amt630hv100 串口升级协议》，相关的代码实现可以参考 Tools\630HV100 串口升级工具\UartUpdate.c 文件。

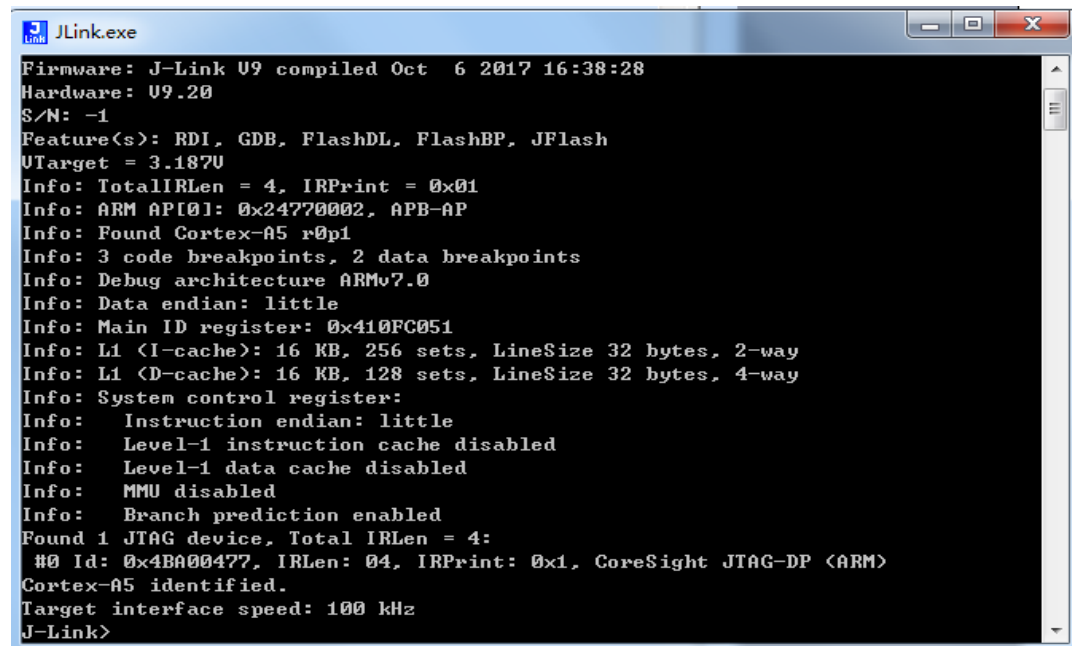
SDK 内的 amt630hv100-freertos 工程和 STEPLDR 工程串口升级默认使用的是 UART1，如果需要使用其他串口，可以修改两个工程里面的 board.h 文件中 UART_MCU_PORT 宏定义的值。

4 JTAG 烧录说明

对于没有引出 sd 卡槽的板子可以使用 JTAG 烧录升级文件。为了防止 sd 卡或者 flash 中存在错误的数据导致上电后运行异常而影响 JTAG 连接，最好此时将启动模式跳到 BIT1(0):BIT0(1) USBDevice 模式。用 IAR 打开 AMTLDR 工程，将 Src/Entry.c 文件里 PROJECT_PURPOSE 宏定义的值改为 PROJECT_FOR_JTAG_UPDATE，重新编译后启动调试，并在下图所示位置添加断点。

```
162 void updateFromJtag(void)
163 {
164     unsigned int loader_addr = 0x20000000;
165     unsigned int stepldr_addr = 0x20010000;
166     unsigned int app_addr = 0x20100000;
167     UpFileHeader *header = (UpFileHeader *)app_addr;
168
169     SendUartString("burn loader start... \r\n");
170     SpiNorBurn((void*)loader_addr, LOADER_OFFSET, LOADER_MAX_SIZE);
171     SendUartString("burn loader end. \r\n");
172
173     SendUartString("burn stepldr start... \r\n");
174     SpiNorBurn((void*)stepldr_addr, STEPLDRA_OFFSET, STEPLDR_MAX_SIZE);
175     SendUartString("burn stepldr end. \r\n");
176
177     SendUartString("burn app start... \r\n");
178     SpiNorBurn((void*)app_addr, IMAGE_OFFSET, header->size);
179     SendUartString("burn app end. \r\n");
180
181     SysInfo *sysinfo = GetSysInfo();
182     sysinfo->app_checksum = header->checksum;
183     sysinfo->stepldr_offset = STEPLDRA_OFFSET;
184     sysinfo->stepldr_size = STEPLDR_MAX_SIZE;
185     sysinfo->update_status = UPDATE_STATUS_END;
186     SaveSysInfo(sysinfo);
187
188     SendUartString("Update is finished. Please reset. \r\n");
189     while(1);
190 }
```

程序运行到此位置后，运行 Jlink.exe 程序，如下图所示：



```
JLink.exe
Firmware: J-Link V9 compiled Oct 6 2017 16:38:28
Hardware: V9.20
S/N: -1
Feature(s): RDI, GDB, FlashDL, FlashBP, JFlash
VTarget = 3.1870
Info: TotalIRLen = 4, IRPrint = 0x01
Info: ARM AP[0]: 0x24770002, APB-AP
Info: Found Cortex-A5 r0p1
Info: 3 code breakpoints, 2 data breakpoints
Info: Debug architecture ARMv7.0
Info: Data endian: little
Info: Main ID register: 0x410FC051
Info: L1 <I-cache>: 16 KB, 256 sets, LineSize 32 bytes, 2-way
Info: L1 <D-cache>: 16 KB, 128 sets, LineSize 32 bytes, 4-way
Info: System control register:
Info: Instruction endian: little
Info: Level-1 instruction cache disabled
Info: Level-1 data cache disabled
Info: MMU disabled
Info: Branch prediction enabled
Found 1 JTAG device, Total IRLen = 4:
#0 Id: 0x4BA00477, IRLen: 04, IRPrint: 0x1, CoreSight JTAG-DP (ARM)
Cortex-A5 identified.
Target interface speed: 100 kHz
J-Link>
```

依次输入以下命令：

speed 100000

```
loadbin D:\projects\amt630h-v100\amt630h-sdk-beta\升级文件\LVGL\spildr.bin
0x20000000
```

```
loadbin D:\projects\amt630h-v100\amt630h-sdk-beta\升级文件\ LVGL\stepldr.bin
0x20010000
```

```
loadbin D:\projects\amt630h-v100\amt630h-sdk-beta\升级文件\ LVGL\update.bin
0x20100000
```

结果如下图所示:

```
J-Link>speed 100000
Maximum target interface speed is 50000 kHz.
Target interface speed: 15000 kHz
J-Link>loadbin D:\projects\630H\amt630h-sdk-beta\升级文件\DDR16X16\spildr.bin 0x
20000000
Downloading file [D:\projects\630H\amt630h-sdk-beta\升级文件\DDR16X16\spildr.bi
n]...O.K.
J-Link>loadbin D:\projects\630H\amt630h-sdk-beta\升级文件\DDR16X16\stepldr.bi
n 0x20010000
Downloading file [D:\projects\630H\amt630h-sdk-beta\升级文件\DDR16X16\stepldr.bi
n]...O.K.
J-Link>loadbin D:\projects\630H\amt630h-sdk-beta\升级文件\DDR16X16\update.bin 0x
20100000
Downloading file [D:\projects\630H\amt630h-sdk-beta\升级文件\DDR16X16\update.bi
n]...O.K.
J-Link>
```

之后继续运行程序到结束打印“Update is finished. Please reset.”。

升级成功后将启动模式设为 BIT1(0):BIT0(0) SD+SPINOR 模式后重新上电。

针对 JLink v4.9 版本的驱动（其他版本未测试），发现此版本 Jlink.exe 程序对文件路径名长度有限制，升级文件的路径名（包括文件名）总长度不能大于 98 个字符（汉字占两个字符），否则上述指令虽然执行成功，但实际上无法把升级文件下载到对应的内存中，最终导致升级后启动失败。

5 升级注意事项

在生产或者调试阶段可以通过 tf 卡或者 jtag 来升级。

由于速度限制，串口和 can 升级支持 amt630hv100.bin, rom.bin, bootanim.bin 单个文件升级，但是 sd 卡和 usb 升级只支持这三个文件制作生成的 update.bin 文件升级。

生产后的机器后续升级使用 sd 卡或者 usb 升级时，不需要将 AMTLDR.bin 文件拷贝到 sd 卡或者 u 盘里。

生产后的机器后续升级通常情况下只需要升级 update.bin 文件或者单个的 amt630hv100.bin, rom.bin, bootanim.bin 文件。没有明确需求，不需要升级 spildr.bin 和 stepldr.bin 文件(如果是 sd 卡或者 usb 升级，则不拷贝这两个文件到 sd 卡或者 u 盘里，如果是串口或者 can 升级则不需要发送这两个文件的升级命令)。

stepldr 程序有备份处理，即使升级 stepldr 异常仍然可以运行该程序的备份，支持后续升级。由于芯片固化程序不支持，spildr 程序没有备份，升级 spildr 异常会导致系统起不来并且不能再通过串口或者 can 或者 usb 升级，只能按照生产阶段的 tf 卡或者 jtag 升级方式来升级。所以再次强调，**生产后到客户终端的机器，没有特别原因，不要升级 spildr.bin 文件。**

升级时候由于要显示进度条需要在 STEPLDR 工程内配置 LCD 模块，STEPLDR 工程里 LCD 的配置和 amt630hv100-freertos 是一致的，两个工程的 board.h 文件里 LCD 相关的参数配置是相同的，默认都是 1024x600 的 TTL (RGB) 点屏，如果使用其他的屏，同步修改两个工程的 board.h 头文件即可。

6 JTAG 调试说明

使用 JTAG 调试工程，为了防止 sd 卡或者 flash 中存在错误的数据导致上电后运行异常而影响 JTAG 连接，最好此时将启动模式跳到 BIT1(0):BIT0(1) USBDevice 模式。

由于 amt630hv100-freertos 应用工程需要下载到 DDR 中调试，因此在调试该应用前需要先下载运行 AMTLDR 工程来初始化 DDR。同样用 IAR 打开 AMTLDR 工程后，将 Src/Entry.c 文件里 PROJECT_PURPOSE 宏定义的值改为 PROJECT_FOR_DDR_INIT，重新编译后点击 Download and Debug 运行，程序跑完后点击 Stop Debugging 退出。之后再运行调试 amt630hv100-freertos 工程。需要注意的是，**调试前确保机器已经升级过(烧录好了动画文件和图片资源文件)，否则可能导致程序运行异常。**

如果觉得上面操作麻烦的话可以将启动模式跳到 BIT1(0):BIT0(0) 模式，保证上电后系统能正常起来的情况下可以直接调试 amt630hv100-freertos 工程。

7 其他说明

- 7.1 SDK 内核 rtos 使用的是 FreeRTOS，相关文档可以查询 FreeRTOS 官方网站 <https://www.freertos.org>。
- 7.2 应用 UI 框架默认使用的是 LVGL，相关文档可以查询 LVGL 官方网站 <https://lvgl.io>。
- 7.3 LVGL 支持中文字体，使用方法参考 Doc 目录下《lvgl 添加中文字体说明》，Demo 应用中 haoke_demo.c 文件里有部分中文字符使用。
- 7.4 AMT630HV100 封装的 DDR2 容量为 32Mx16(64MB)。
- 7.5 芯片的管脚复用在 pinctrl.c 文件里配置，可以根据自己的产品板的管脚使用不同，在 vPinctrlSetup 函数配置正确的管脚使用。

- 7.6 大部分管脚都可以设置驱动能力，比如要修改 LCD DCLK 信号的驱动强度，可以先查看芯片原理图找到 LCD_CLK 和 I089 是复用的，之后在 pinctrl.c 文件 pin_groups 数组里找到 LCD 的管脚配置中对应 I089 的部分，在后面添加想要设置的驱动电流，比如 PAD_DRIVE_2MA，未添加驱动电流设置的管脚使用的是默认值 PAD_DRIVE_8MA。

```
{.groupid = PGRP_LCD_TTL_CH0, .pins_num = 28,
 /* de      clk      vvnsc      hsync */
 .pins = {{88, 1}, {89, 1}, {PAD_DRIVE_2MA}, {90, 1}, {91, 1},
 {64, 1}, {65, 1}, {66, 1}, {67, 1}, {68, 1}, {69, 1}, {70, 1}, {71, 1}, /* r0-r7 */
 {72, 1}, {73, 1}, {74, 1}, {75, 1}, {76, 1}, {77, 1}, {78, 1}, {79, 1}, /* g0-g7 */
 {80, 1}, {81, 1}, {82, 1}, {83, 1}, {84, 1}, {85, 1}, {86, 1}, {87, 1}}, /* b0-b7 */
```

- 7.7 由于 PWM 脚的管脚复用比较多，每个客户使用的管脚也不同，因而在代码上不好完全统一，需要用户自己修改一下。举个具体的例子，假如你需要使用 PWM2，并且使用的是芯片 47 脚的 PWM2，我们可以看到芯片原理图 47 脚是和 I06 复用的，



所以修改 pinctrl.c 文件下面行，将 {2, 1} 改成 {6, 1}。

```
{.groupid = PGRP_PWM2, .pins_num = 1, .pins = {{2, 1}}},
```

另外在 vPinctrlSetup 函数里添加 pinctrl_set_group(PGRP_PWM2)。

- 7.8 Demo 应用默认没有打开倒车显示，需要的话修改 amt630hv100-freertos\app\main_lvgl.c 文件，将注释掉的 carback_init() 调用打开。此时会创建了一个 carback_test_thread 线程来模拟发送进入和退出倒车消息，大约每隔 30s 会进入倒车显示，10s 后退出。Demo 程序配置的 RN6752 支持 AHD 720P 和 CVBS 信号，如果板上的 AHD 输入座接入了正确的 AHD/CVBS 信号，在进入倒车的时候可以看到视频图像，如果没有接入信号的话此时则会显示蓝屏。

- 7.9 串口通信可以参考 Demo 应用内的 uart_rx_demo_thread 线程。

- 7.10 Can 通信可以参考 Demo 应用内的 can_demo_thread 线程。

- 7.11 支持 SD 卡，默认是打开的，如果不需要可以修改 board.h 头文件将 SDMMC_SUPPORT 宏定义注释掉。打开 SD 卡支持时，开启宏定义 SDMMC_TEST 会创建 sdcard_read_thread 线程来检测插入的卡里面是否包含有 update.bin 升级文件，并且会判断该文件的版本和已经烧录到 flash 里的版本是否一致，不一致的话就会重启机器进行 SD 卡升级。

- 7.12 支持 U 盘升级，默认是关闭的，如果需要可以修改 board.h 头文件将 USB_SUPPORT 宏定义打开。打开 U 盘升级支持时会创建 usb_read_thread 线程来检测插入的 U 盘里面是否包含有 update.bin 升级文件，并且会判断该文件的版

本和已经烧录到 flash 里的版本是否一致，不一致的话就会重启机器进行 U 盘升级。

- 7.13 支持 USB UVC 摄像头，默认是关闭的，如果需要可以修改 board.h 头文件将 USB_UVC_SUPPORT 宏定义打开。打开 UVC 摄像头支持时插入 USB 摄像头后会自动显示摄像头图像，拔出则返回到 UI 界面。
- 7.14 支持电阻触摸屏，不过默认是关闭的。如果需要测试电阻触摸屏的话可以修改 board.h 头文件将注释掉的 ADC_TOUCH 宏定义打开，另外修改 lv_ex_conf.h 头文件将 LV_USE_DEMO_PRINTER 宏定义的值改为 1。
- 7.15 支持 GT9XX 系列电容触摸屏，不过默认是关闭的。如果需要测试电容触摸屏的话同样修改 board.h 头文件将注释掉的 GT9XX_TOUCH 宏定义打开，另外修改 lv_ex_conf.h 头文件将 LV_USE_DEMO_PRINTER 宏定义的值改为 1。GT9XX 触摸屏驱动不支持固件下载升级，触摸屏 x 和 y 方向坐标点是否需要翻转可以通过打开 GT9XX_INVERTED_X 和 GT9XX_INVERTED_Y 宏定义来配置。
- 7.16 支持 ADC 按键，默认是打开的，如果不需要可以修改 board.h 头文件将 ADC_KEY 宏定义注释掉。打开 ADC 按键支持，按下 Demo 板上的不同 ADC 按键时，调试串口会输出 keypad_input_read xx press/release。
- 7.17 支持 RTC，默认是打开的，如果不需要可以修改 board.h 头文件将 RTC_SUPPORT 宏定义注释掉。调用 vSetLocalTime 一次设置好时间后，只要 RTC 不掉电，后续调用 iGetLocalTime 可以获取当前时间。
- 7.18 支持 TTL, LVDS, CPU, MIPI 点屏接口，默认设置的是 1024x600 的 TTL 屏, 使用其他屏可以在 board.h 头文件里修改屏相关的各种参数。
- 7.19 支持 RN6752, ARK7116 视频解码芯片，具体使用哪个也可以在 board.h 头文件里配置。ARK7116 只支持 CVBS 信号，RN6752 同时支持 CVBS 和 AHD，可以在 board.h 头文件配置和使用的摄像头匹配的格式。
- 7.20 支持 spi norflash 四线模式，默认是关闭的，如果需要可以修改 board.h 头文件将 SPI0_QSPI_MODE 宏定义打开。amt630hv100-freertos 和 stepldr 工程可以分别配置是否需要。由于不同的 norflash 芯片 4 线模式的操作命令可能存在差异，暂时只调试测试了 Winbond 的部分 norflash 芯片。
- 7.21 支持屏幕旋转（横屏竖用或者竖屏横用），修改 board.h 头文件 LCD_ROTATE_ANGLE 宏定义的值设置需要旋转的角度。需要注意的是旋转只支持源数据的宽度 stride(width*bpp/8)值小于 4096。
- 7.22 支持屏幕水平或者垂直镜像显示，修改 board.h 头文件 LCD_H_FLIP 以及 LCD_V_FLIP 宏定义的值 1 打开相应的功能。



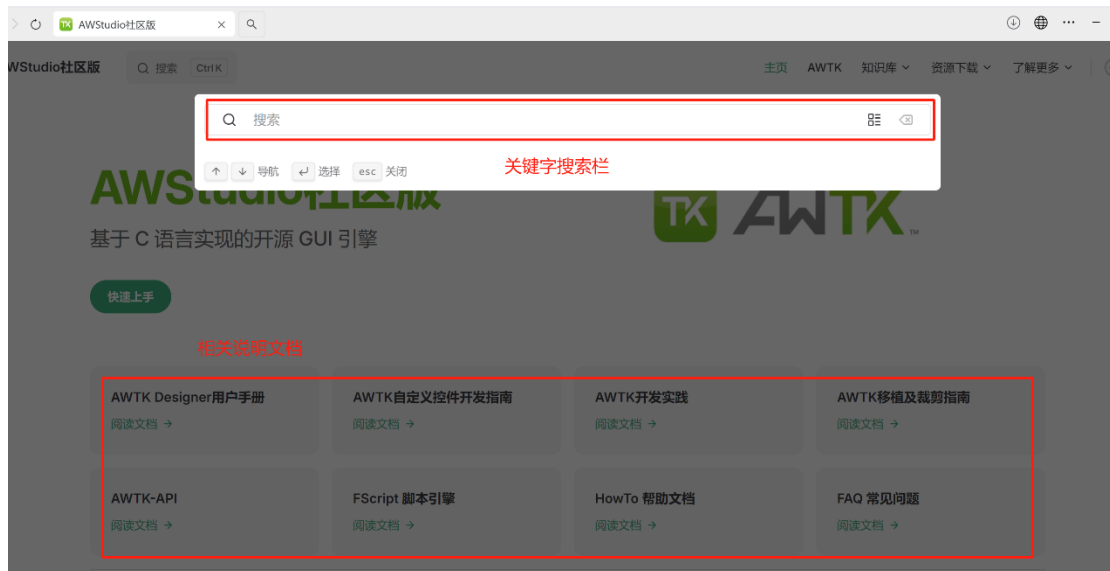
- 7.23 支持 wav 音频播放，默认是关闭的，如果需要可以修改 board.h 头文件将 AUDIO_REPLAY 宏定义打开。
- 7.24 支持 I2C slave 模式，验证该功能可以将两块开发板的 I2C0 飞线连接，一块板修改 board.h 将 I2C_EEPROM_MASTER_DEMO 宏定义打开作为 master，另一块板修改 board.h 将 I2C0_SLAVE_MODE 和 I2C_EEPROM_SLAVE_DEMO 宏定义打开作为 slave。slave 开发板不掉电情况下可以模拟 eeprom24c02 功能供 master 开发板读写。
- 7.25 amt630hv100-freertos 工程默认支持 vg_driver，可以通过 openVG API 来使用硬件加速绘制 2D 图像，如果不需要的话需要修改工程设置，删除 VG_DRIVER 符号定义。
- 7.26 若使用 VG_DRIVER，所有 VG 相关的函数调用必须都放到 xm_vg_loop 函数内。Demo 应用做了一个简单的 lv_openvg 控件来控制 VG 的绘制。现在使用 VG 绘图的基本思路是在屏上划出一块区域，该区域内所有内容都交给 VG 去绘制。使用 VG 绘图需要 openVG 编程基础。
- 7.27 amt630hv100-freertos 工程提供完全由 openvg 来绘制界面的 demo，该工程除了 LVGL 工程配置外还有 VG_ONLY 工程配置，如果只需要使用 openvg 而不需要 lvgl 的话可以选择 VG_ONLY 工程配置。我们提供了单表盘和双表盘两种 Demo，在工程的符号定义中添加 DOUBLE_POINTER_HALO 定义则绘制双表盘 demo，该 demo 需要屏幕的分辨率为 1280x480，具体的绘制过程和绘制方法以及字体和图片的修改在 double_pointer_demo.c 文件内有详细说明，用到的图片和字体资源都在 double_pointer_halo 目录下。在工程的符号定义中添加 SINGLE_POINTER_HALO 定义则绘制单表盘 demo，该 demo 需要屏幕的分辨率为 1024x600，具体的绘制过程和绘制方法以及字体和图片的修改在 single_pointer_demo.c 文件内有详细说明，用到的图片和字体资源都在 double_pointer_halo 目录下。选择 VG_ONLY 时不需要 lvgl，可以将 amt630hv100-freertos\lib\LittlevGL 目录移除。
- 7.28 支持在倒车视频上叠加显示界面，如果需要的话需要修改工程设置，添加 REVERSE_UI 符号定义，此时会设置 LCD_BPP 为 32。另外，如果需要在倒车视频上叠加动态倒车轨迹，需要修改工程设置，添加 REVERSE_TRACK 定义。倒车轨迹需要使用 openvg 来绘制，所以同时也要有 VG_DRIVER 定义。VG_DRIVER，REVERSE_TRACK，REVERSE_UI 这些符号定义的修改要同步修改 amt630hv100 和 lvgl 两个子工程。现有的版本暂不支持正常界面和倒车轨迹都使用 openvg。
- 7.29 Demo 应用默认打开了图像刷新帧率监测，会在屏幕右下角显示刷新帧率和 CPU 使用率，如果需要去掉的话修改 lv_conf.h 头文件，将 LV_USE_PERF_MONITOR 宏

定义的值改为 0。

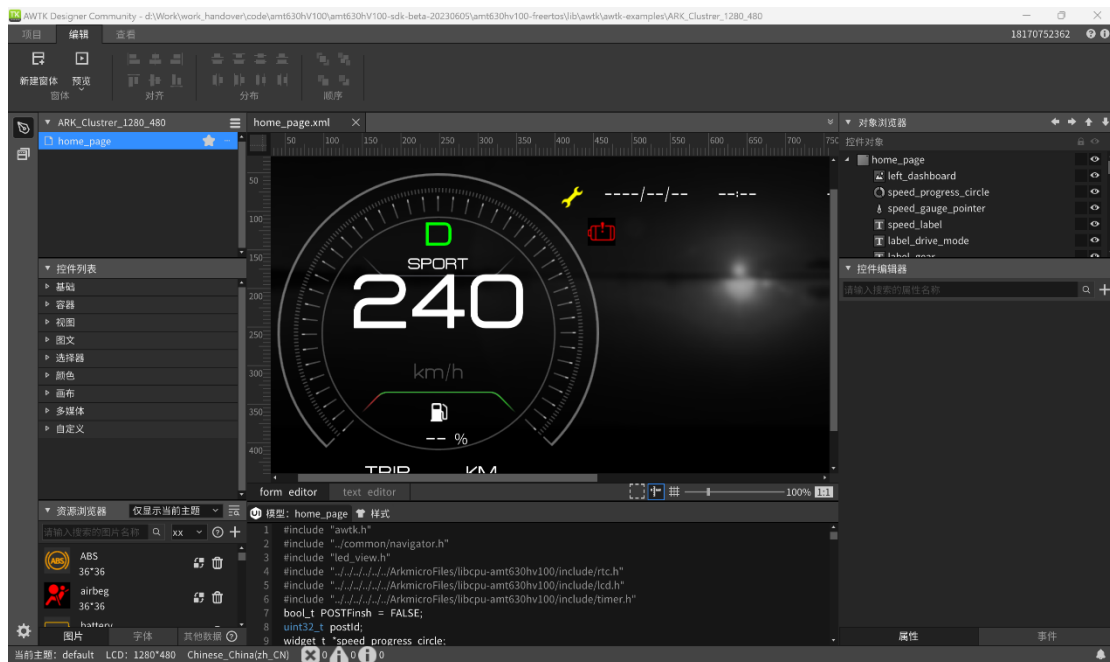
7.30 如果需要打印 Demo 应用每个线程的状态信息，需要在 main_lvgl.c 或者 main_openvg.c 文件添加 TASK_STATUS_MONITOR 宏定义，该打印会影响界面刷新效率，量产软件需要去掉该打印。

7.31 awtk 安装使用及 API 索引，初学可以以下链接为教程进行学习索引：

<https://awtk.zlg.cn/docs/>

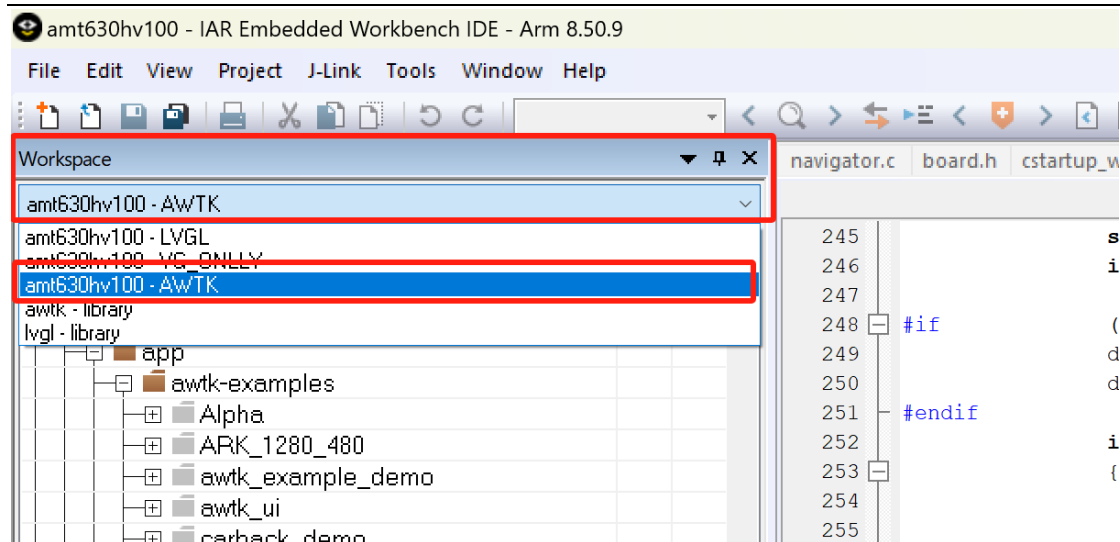


创建 awtk designer 工程，并对其进行布局，ui 设计，程序逻辑开发，为 app 工程。
App 工程路径推荐生成在此目录下：amt630hv100-freertos\lib\awtk\awtk-examples

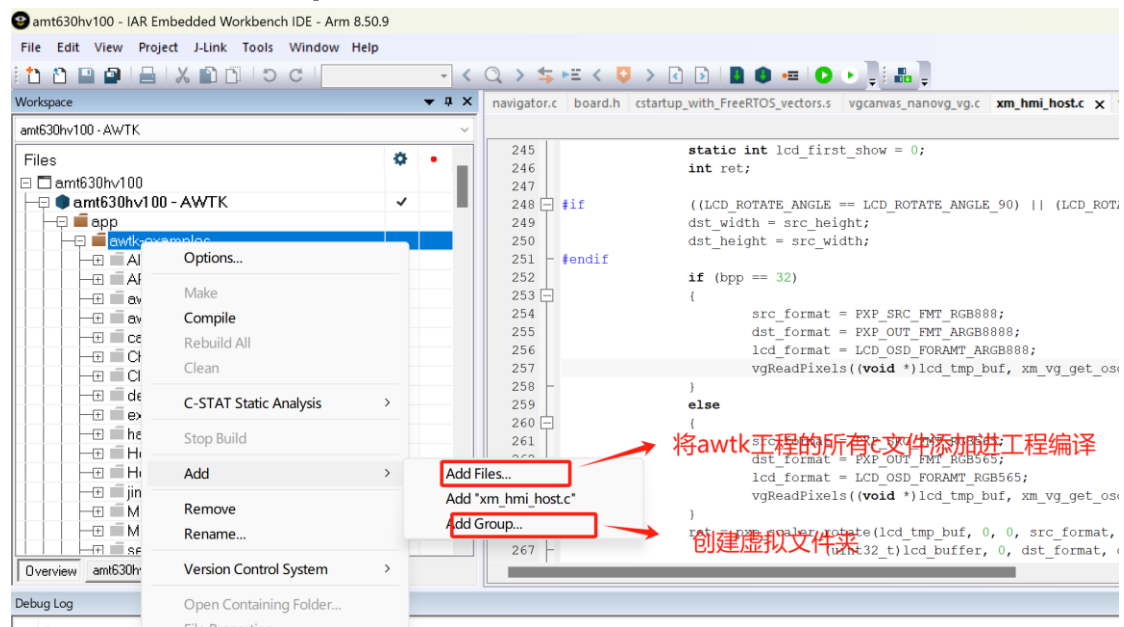


将已经设计好的 awtk 工程项目添加进 IAR 工程里编译。步骤如下：

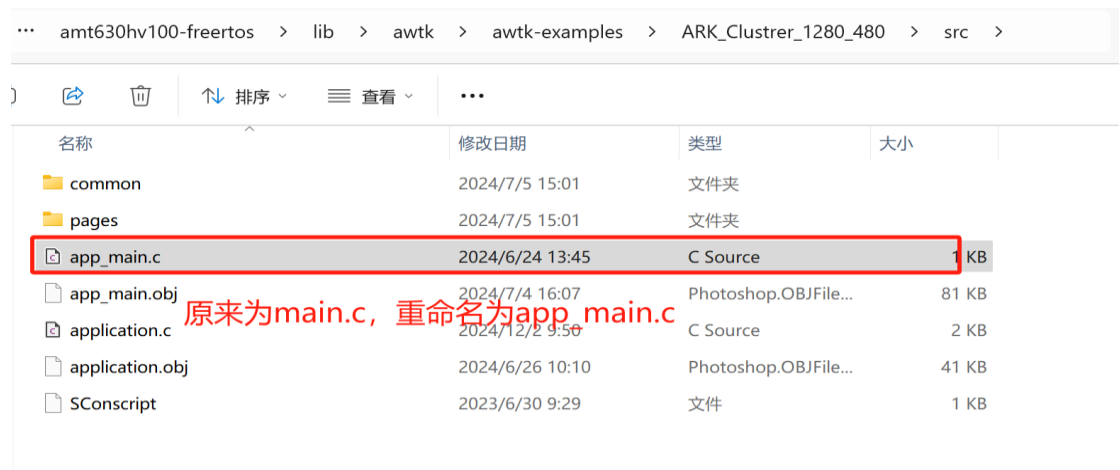
② 将 IAR 的 workspace 切换至 awtk 版本；



② 在 awtk 版本的 workspace 下添加虚拟路径，导入我们一开始创建完成的 awtk 工程；



③ 将 awtk 工程下的 main.c 文件重命名修改为 app_main.c 文件，避免与 IAR 工程下的 main.c 文件冲突；

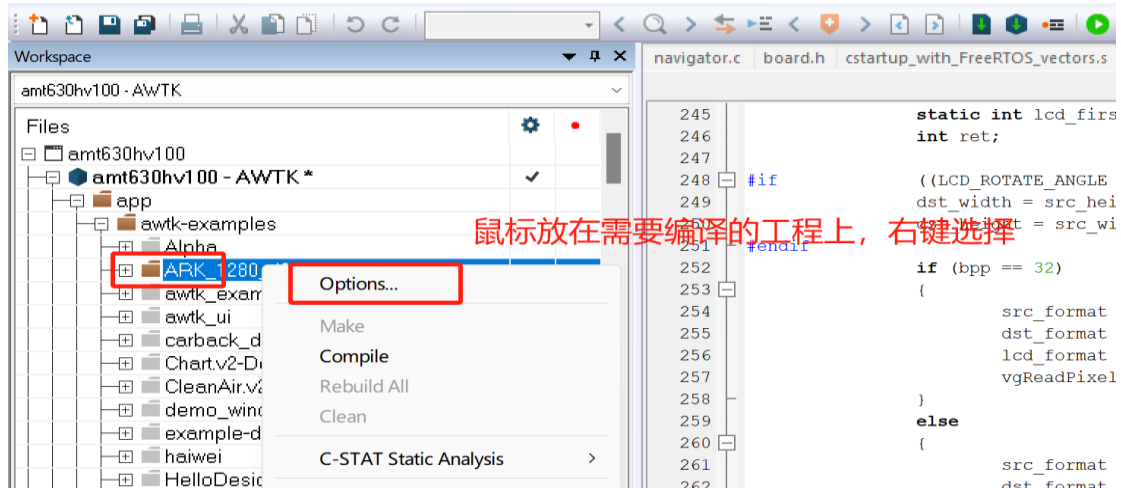


③ 需导入的文件所在文件夹如下：

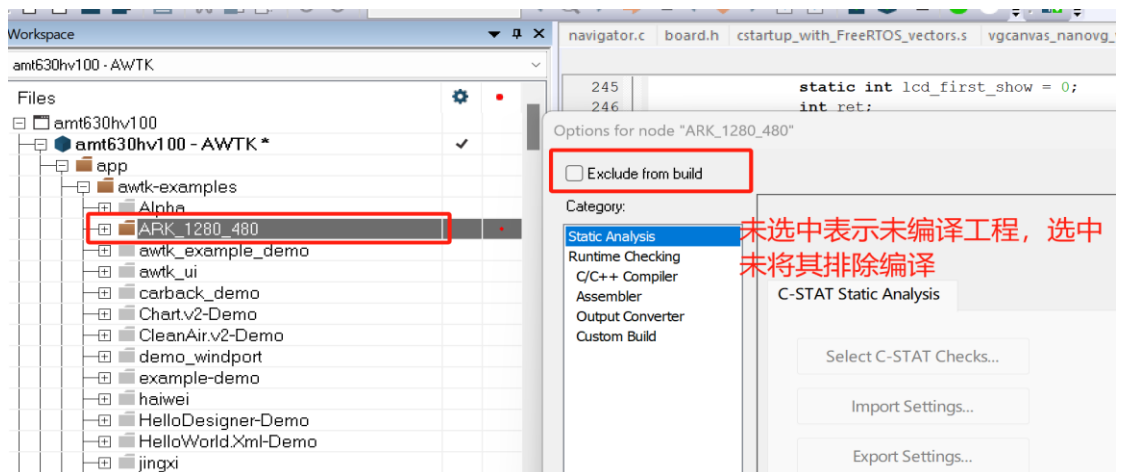


按以下步骤修改选择编译的 awtk 工程，awtk-examples 中只留其中一项为深褐色，深褐色为可编译过程，只能选择一个 awtk 工程进行编译。

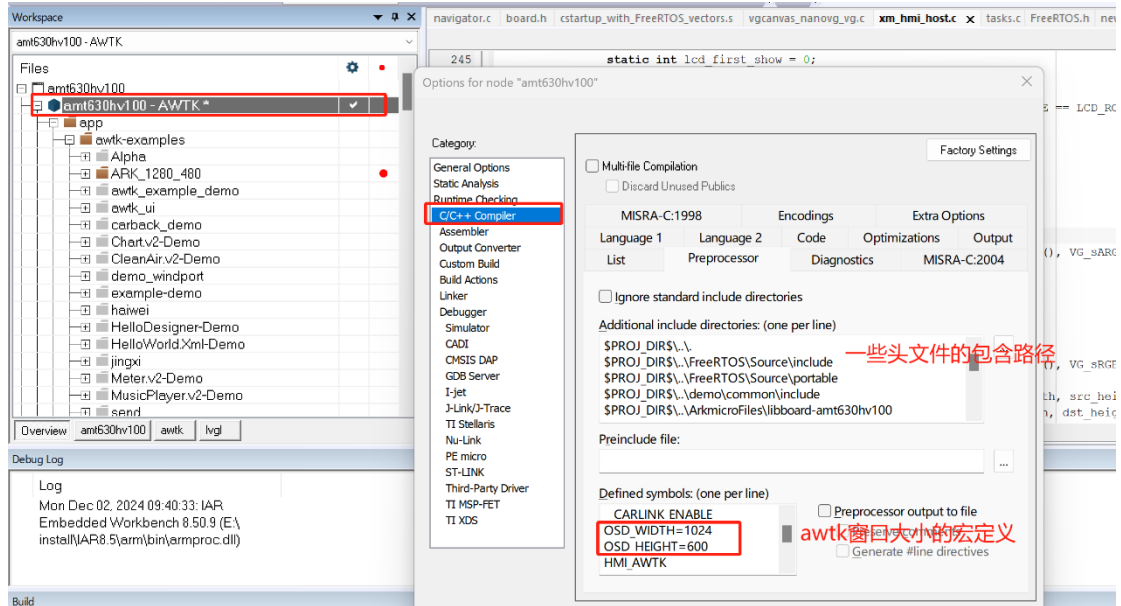
① 鼠标放在需编译的工程上，右键选择“options”进行设置



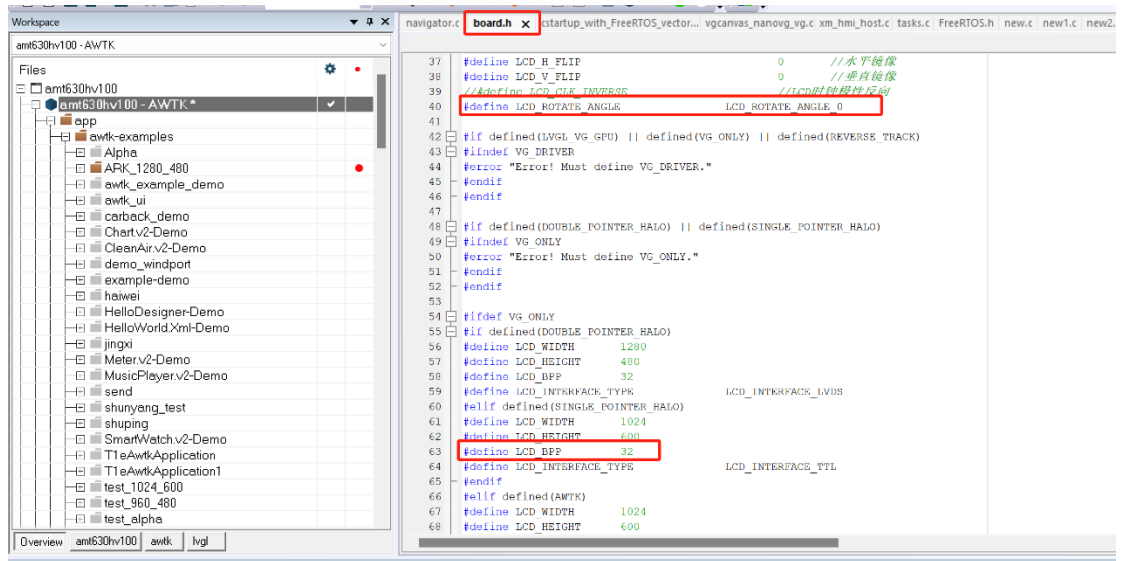
② Options 弹窗左上角有“Exclude from build”选择，选中为移除工程编译，未选中为活动工程，添加进工程编译。



设置 awtk 工程相关的宏定义及头文件路径：

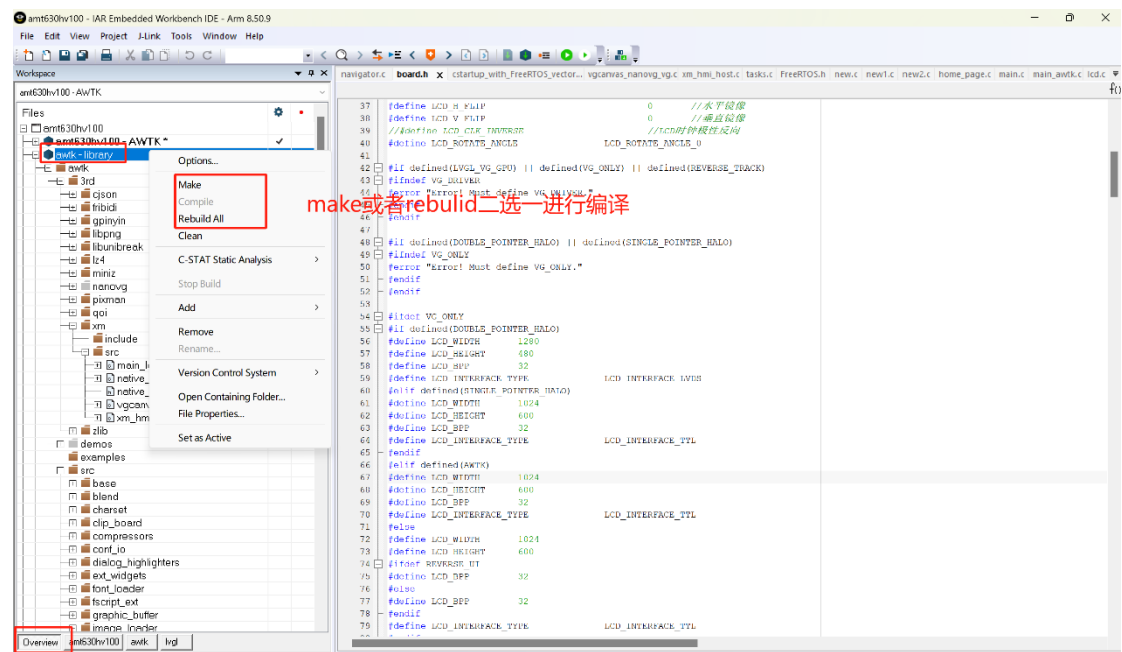


查看 board.h ，修改旋转或者是显示格式为 16 还是 32bpp，都需要切换到对应的 awtk 库环境下进行 库的编译更新。

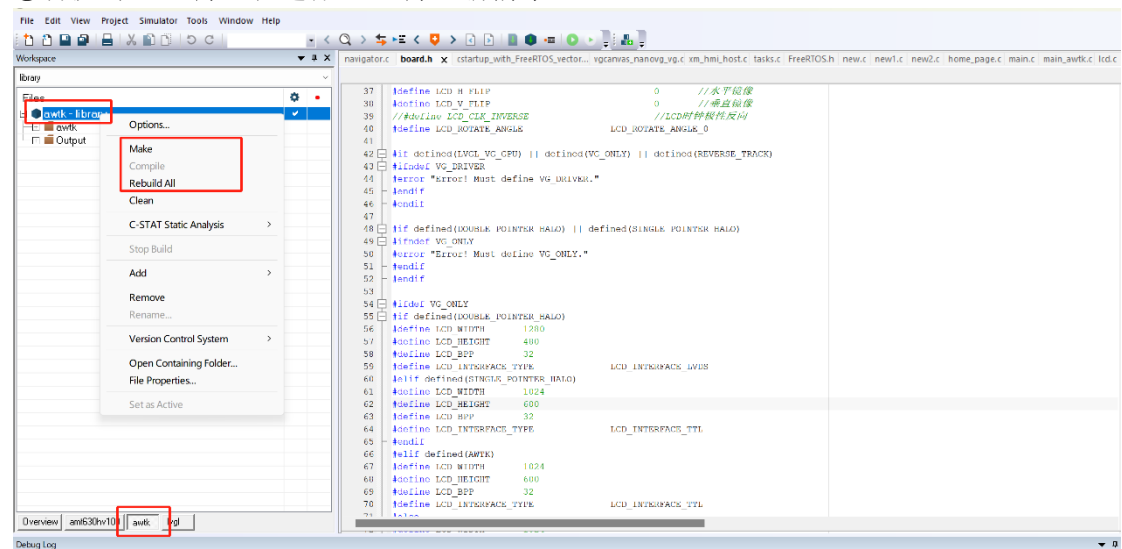


修改库相关文件或者 board.h 上述文件，需要重新编译 awtk 库。如下：

- ① 在 overview 工程下，选择 awtk-library 工程进行 awtk 库重新编译；



②切换到 awtk 库工程进行 awtk 库重新编译



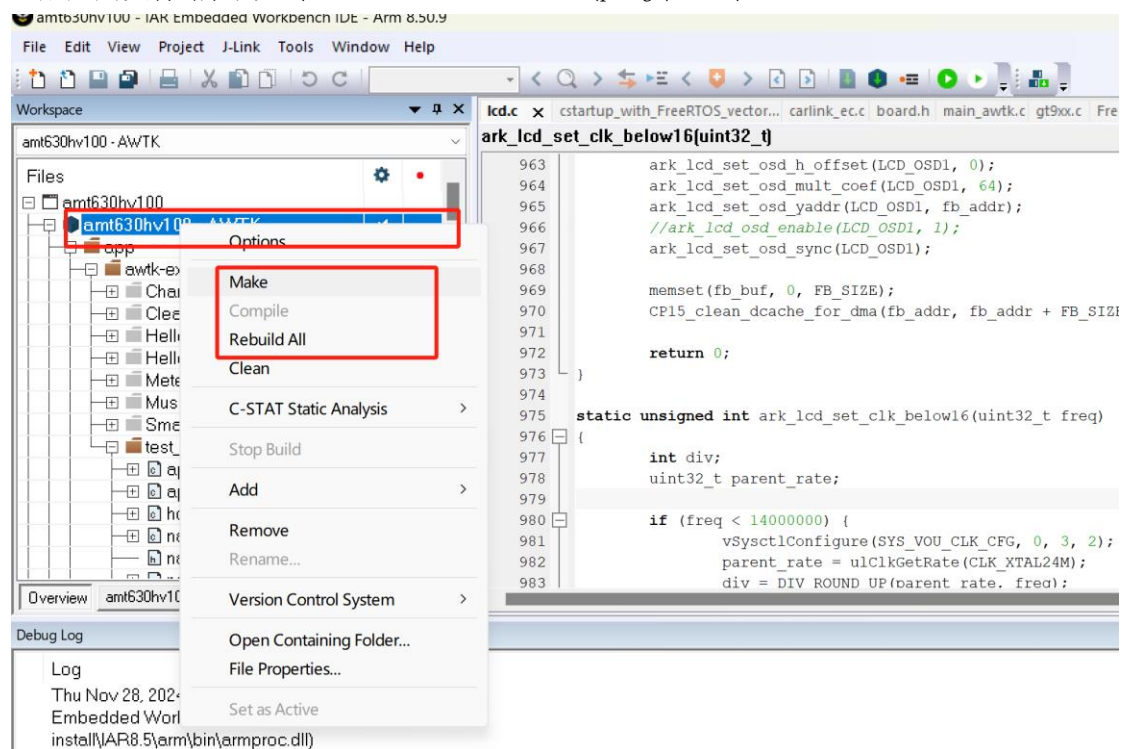
③ 编译完成后会重新生成 awtk 库文件 awtk_rgb565. a

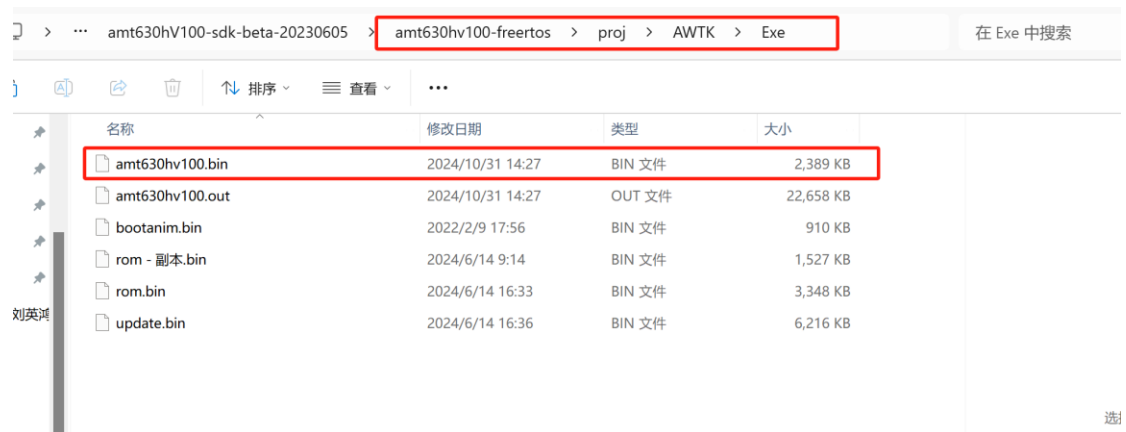
... amt630hV100 > amt630hV100-sdk-beta-20230605 > amt630hv100-freertos > lib > awtk > 在 av

名称	修改日期	类型	大小
.vscode	2024/3/5 9:48	文件夹	
awtk	2024/5/15 13:46	文件夹	
awtk-examples	2024/11/14 18:05	文件夹	
awtk-msvc	2023/12/25 17:16	文件夹	
awtk-mvvm	2023/12/26 10:42	文件夹	
awtk_rgb565.a	2024/10/31 11:37	A 文件	31,362 KB
nanovg - 副本.a	2022/4/1 15:56	A 文件	1,204 KB
nanovg.a	2024/2/20 9:08	A 文件	1,208 KB
nanovg_1.a	2023/12/28 14:01	A 文件	1,205 KB

8、切回 amt630hv100 工程，重新编译 amt630hv100 工程，生成烧录应用文件 amt630hv100.bin 文件。

生成应用文件路径为：\amt630hv100-freertos\proj\AWTK\Exe



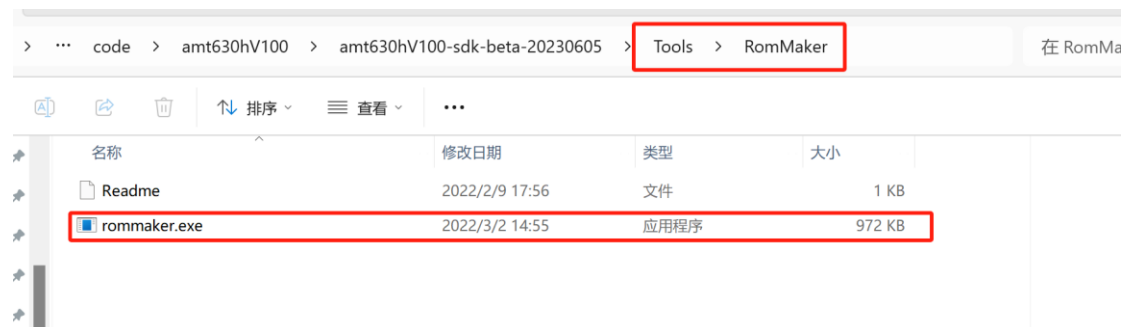


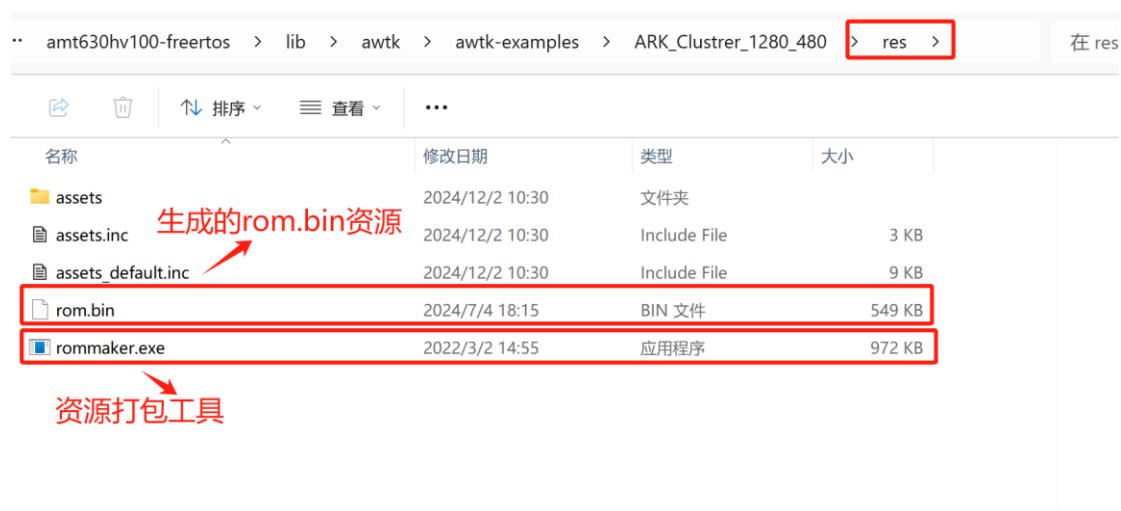
9、awtk 应用资源打包。

①Awtk 工程编译完成后会生成res文件夹，进入res文件夹，该文件夹主要有两部分awtk-examples\ARK_Clustrer_1280_480\res\assets\default\inc 文件夹和 awtk-examples\ARK_Clustrer_1280_480\res\assets\default\raw 文件夹。inc 文件夹下的资源未使用到，且文件较大会占用很大的控件，故打包资源前需要删除 inc 文件夹。

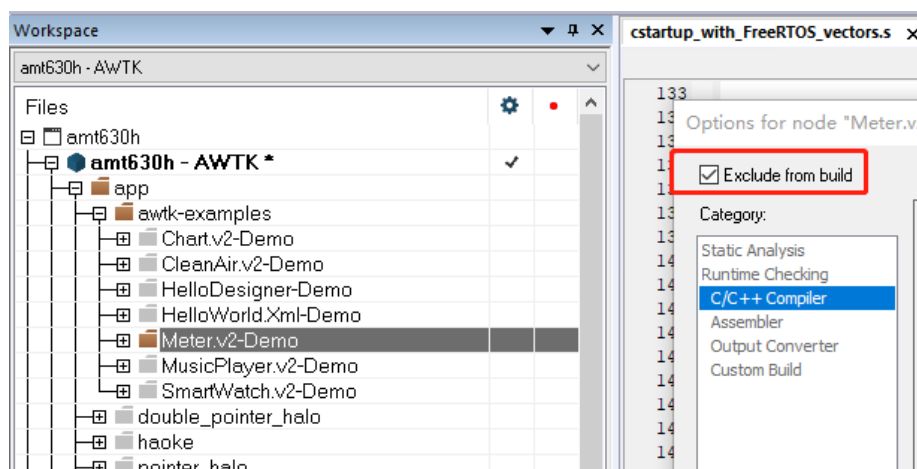


② 删除 inc 文件加后，将 SDK 下 Tools\RomMaker 文件下的 rommaker.exe 拷贝到 res 文件夹下，点击运行会自动读取同级目录下的所有资源打包生成 rom.bin 文件。





- 7.32 支持 AWTK UI 引擎，编译 AWTK 应用需要选择 AWTK 工程配置，工程包含 7 个 AWTK 的官方示例程序，默认选择的是 Meter.v2-Demo，如果需要运行其他的示例程序需要修改工程配置，把 Meter.v2-Demo 代码从编译中排除，之后把其他的 Demo 代码包含到编译中。



另外注意不同的 Demo 需要升级相对应的资源文件。生成资源文件镜像时可以将 RomMaker.exe 文件拷贝到 awtk 工程对应的 res 目录下，之后将 assets/inc 目录以及**/*.inc 文件移除(这些文件在没有定义 WITH_FS_RES 而将资源文件编译到应用本身时会使用到)，再双击运行 RomMaker.exe 生成 rom.bin 文件。

- 7.33 为了优化 AWTK 内存使用，添加了 AWTK 单独的堆大小设置，通过修改 platform.c 文件 AWTK_HEAP_SIZE 宏定义来改变。但是需要注意的是，修改该值后需要同步修改 FreeRTOSConfig.h 文件 AWTK 对应的 configTOTAL_HEAP_SIZE 宏定义的值，需要保证两个宏定义的值和是不变的。

- 7.34 支持尔雅和亿联手机互联，默认已打开，可以按照下述操作进行关闭：在对应

工程的配置选项（options->C/C++ Compiler->Preprocessor）中屏蔽宏定义 CARLINK_ENABLE，并在工程中取消包含如下文件以减小编译镜像大小（app/carlink、FreeRTOS-Plus/FreeRTOS-Plus-TCP、lib/bt、lib/wlan）。领骑手机互联使用说明参考“Doc/领骑摩托使用说明书.pdf”。

- 7.35 支持 Spi Nand Flash。系统默认配置是 Spi Nor Flash，可以根据需求切换为 Spi Nand Flash，详细使用说明请参考“Doc/amt630hv100 SPI NAND FLASH 使用说明.pdf”。
- 7.36 支持 EMMC。系统默认配置是 Spi Nor Flash，可以根据需求切换为 Spi Nand Flash，步骤：把 AMTLDR 和 STEPLDR 两个工程下的 amt630h.h 和 amt630hv100-freertos 工程下的 board.h（共三个头文件）里的宏定义 DEVICE_TYPE_SELECT 配置为 EMMC_FLASH 即可。另外使用 emmc 的时候增多一个 lnchemmc.bin 升级文件用来烧录到启动的 norflash 内来引导 emmc 内的 loader 程序。另外 spildr.bin 文件改名为 emmcldr.bin 文件。AMTLDR 工程 PROJECT_PURPOSE 宏定义值为 PROJECT_FOR_LAUNCH_EMMC 时将编译生成的 amtldr.bin 文件改名为 lnchemmc.bin。值为 PROJECT_FOR_EMMC_LOADER 时将编译生成的 amtldr.bin 改名为 emmcldr.bin，需要特别注意的是，**此时需要修改链接的 icf 配置文件，由 AMTLDR.icf 替换为 EMMCCLR.icf**。使用 JTAG 升级 EMMC 时相对于 SPI FLASH 需要多升级一个 lnchemmc.bin 文件下载到 0x20008000 地址，因而文件下载存在一些差异，如下图所示：

```
J-Link>speed 100000
Maximum target interface speed is 50000 kHz.
Target interface speed: 15000 kHz
J-Link>loadbin D:\projects\amt630h-v100\amt630hV100-sdk-beta\升级文件\emmcldr.bin 0x20000000
Downloading file [D:\projects\amt630h-v100\amt630hV100-sdk-beta\升级文件\emmcldr.bin]...O.K.
J-Link>loadbin D:\projects\amt630h-v100\amt630hV100-sdk-beta\升级文件\lnchemmc.bin 0x20008000
Downloading file [D:\projects\amt630h-v100\amt630hV100-sdk-beta\升级文件\lnchemmc.bin]...O.K.
J-Link>loadbin D:\projects\amt630h-v100\amt630hV100-sdk-beta\升级文件\stepldr.bin 0x20010000
Downloading file [D:\projects\amt630h-v100\amt630hV100-sdk-beta\升级文件\stepldr.bin]...O.K.
J-Link>loadbin D:\projects\amt630h-v100\amt630hV100-sdk-beta\升级文件\update.bin 0x20100000
Downloading file [D:\projects\amt630h-v100\amt630hV100-sdk-beta\升级文件\update.bin]...O.K.
J-Link>
```

- 7.37 添加 ULOG 日志管理。修改 board.h 头文件添加 USE_ULOG 宏定义打开该功能。此时程序里面使用 TRACE_X 的接口会用到日志管理系统，比如 TRACE_INFO，TRACE_ERROR。

ULOG_BACKEND_USING_CONSOLE 宏打开表示日志输出到串口；

ULOG_EASYFLASH_BACKEND_ENABLE 宏打开表示日志保存到 spiflash 或者 emmc；

ULOG_FILE_BACKEND_ENABLE 宏打开表示日志保存到文件系统；

另外一些相关的宏定义的意义可以参照宏定义的名称。

- 7.38 添加 OTA 升级功能。如果存储设备有足够的空间保证 update.bin 文件做 A/B 备

份的话可以修改 board.h 头文件添加 OTA_UPDATE_SUPPORT 宏定义打开该功能。
此时串口、SD 卡、USB 等升级都直接在应用工程烧写备份分区，而不会再重启到
stepldr 工程完成升级，具体使用可以参考 main_awtk.c 文件里面示例。

7.39 添加 NCM 功能。默认关闭 NCM 功能，相关使用和测试步骤请参考
“Doc/amt630hv100 NCM 模式使用说明.doc” 文件。

7.40 添加差分升级功能，默认关闭。相关使用说明参考 “Doc/amt630hv100 差分升
级使用说明.pdf”

备注：

1. 文中出现的一些代码文件完整路径如下：

amt630hv100-freertos\ArkmicroFiles\libboard-amt630hv100\include\board.h

amt630hv100-freertos\ArkmicroFiles\libcpu- amt630hv100\source\pinctrl.c

amt630hv100-freertos\app\double_pointer_halo\double_pointer_demo.c

amt630hv100-freertos\lib\LittlevGL\lv_ex_conf.h

amt630hv100-freertos\lib\LittlevGL\lv_conf.h

amt630hv100-freertos\app\double_pointer_halo\double_pointer_halo.c

amt630hv100-freertos\app\single_pointer_halo\single_pointer_halo.c

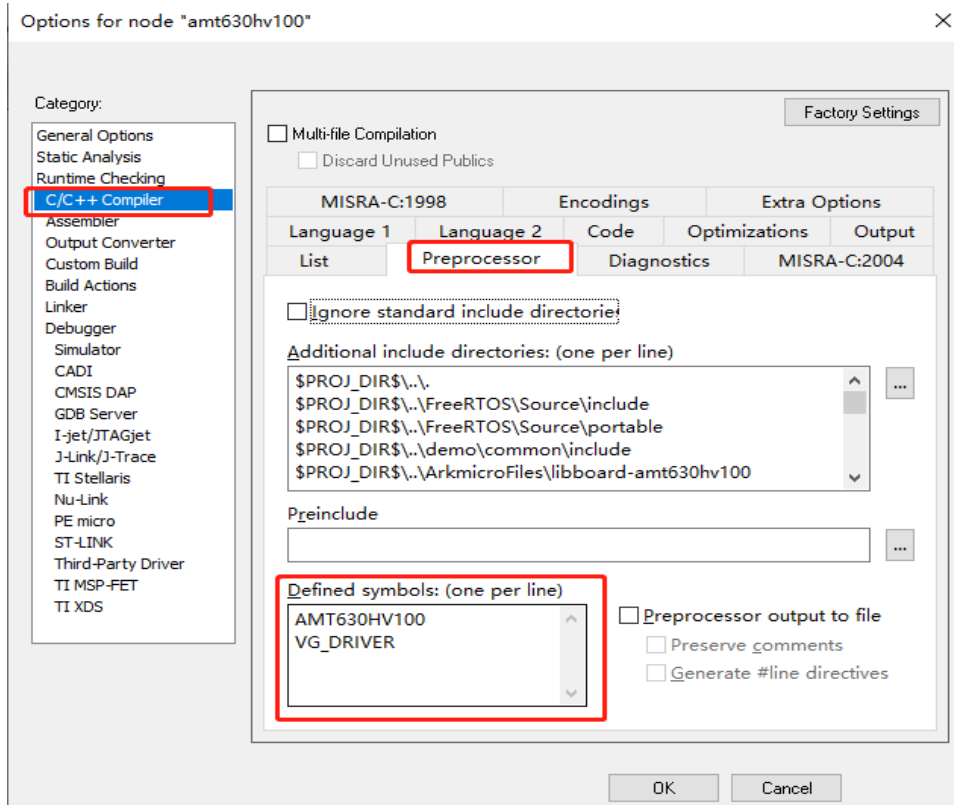
amt630hv100-freertos\app\main_lvgl.c

amt630hv100-freertos\app\main_openvg.c

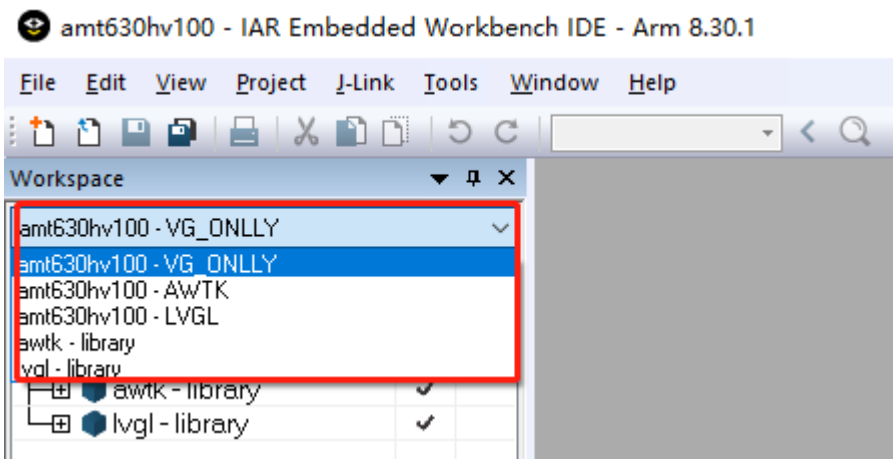
amt630hv100-freertos\FreeRTOSConfig.h

amt630hv100-freertos\lib\awtk\awtk\src\platforms\ark\ platform.c

2. 文中提到的修改工程设置，添加或者删除某个符号定义是指下图所示位置添加或者删
除。



3. 文中提到的选择工程配置按照下图所示位置来选择



4. amt630hv100-freertos 包含三个子工程，amt630hv100 应用工程，awtk 库工程和 lvgl 库工程。amt630hv100 工程包含几个工程配置，不同的工程配置可能需要依赖对应的 awtk 或者 lvgl 库。所以，如果修改的是 awtk 或者 lvgl 库工程的源码需要先编译库工程再重新编译 amt630hv100 应用工程，或者可以创建 batch build 项。点击 Project->Batch build... (快捷键 F8) 可以看到已创建的 amt630hv100-awtk 和 amt630hv100-lvgl 两个编译项。选择 amt630hv100-lvgl 点击 Edit 可以看到该项会先编译 lvgl 工程再编译 amt630hv100 工程的 LVGL 配置。这个时候只要点击 Batch Build 弹出框里的 Make/Clean/Rebuild All 就可以按顺序编译两个工程。

